

INFO145 - Diseño y Análisis de Algoritmos

Análisis Amortizado - Método del Potencial

Héctor Ferrada

académico

Instituto de Informática
Universidad Austral de Chile

1. Análisis Amortizado - Método del Potencial

El análisis amortizado mediante el **método del potencial** es una técnica para evaluar el costo promedio de una secuencia de operaciones sobre una estructura de datos, garantizando una cota superior más ajustada que el análisis del peor caso para cada operación individual.

1.1. Funcionamiento del método del potencial

1. Concepto básico

Se asocia a cada estado D_i de la estructura de datos una función potencial $\Phi(D_i)$, que representa una “energía almacenada” o “crédito” acumulado por operaciones previas. Esta energía puede ser usada para pagar operaciones costosas futuras.

2. Definición formal

Sea c_i el costo real de la i -ésima operación. La estructura pasa del estado D_{i-1} al estado D_i tras esta operación. La función potencial Φ asigna un valor real a cada estado. El costo amortizado $\hat{c}_i$ de la operación i se define como:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

Esto significa que el costo amortizado es el costo real más el cambio en la energía potencial.

3. Interpretación

- Si $\Phi(D_i) > \Phi(D_{i-1})$, la operación incrementa la energía potencial, es decir, “ahorra” trabajo para operaciones futuras (costo amortizado mayor que el real).
- Si $\Phi(D_i) < \Phi(D_{i-1})$, la operación consume energía almacenada, pagando parte de su costo con ese crédito acumulado (costo amortizado menor que el real).

4. Cota superior del costo total

Sumando los costos amortizados de una secuencia de N operaciones:

$$\sum_{i=1}^N \hat{c}_i = \sum_{i=1}^N c_i + \Phi(D_N) - \Phi(D_0)$$

Si se elige la función potencial de modo que $\Phi(D_N) \geq \Phi(D_0)$, entonces:

$$\sum_{i=1}^N \hat{c}_i \geq \sum_{i=1}^N c_i$$

Por lo tanto, la suma de costos amortizados es una cota superior al costo real total, asegurando que el análisis amortizado no subestima el costo real.

5. Ventajas

- Permite distribuir el costo de operaciones caras a lo largo de varias operaciones más baratas.
- Resulta en una cota más ajustada que multiplicar el peor caso individual por el número de operaciones.
- Es especialmente útil en estructuras de datos dinámicas como pilas extendidas, tablas dinámicas, contadores binarios, etc.

Resumiendo, el método del potencial en análisis amortizado consiste en:

- Definir una función potencial que mide el “crédito acumulado” en la estructura.
- Calcular el costo amortizado de cada operación como su costo real más el cambio en la función potencial.
- Mostrar que la función potencial nunca disminuye por debajo de su valor inicial para garantizar que la suma de costos amortizados sea una cota superior del costo real total.

Esto permite analizar secuencias de operaciones con costos variables y obtener una estimación más realista y útil para el diseño y análisis de algoritmos y estructuras de datos avanzadas.

1.2. Ejemplo Ilustrativo: Pila Extendida

Consideremos una pila con las siguientes operaciones:

- **Push (Apilar):** inserta un elemento en la pila.
- **Pop (Desapilar):** elimina el elemento superior.
- **MultiPop (DesapilarMúltiple(k)):** elimina los k elementos superiores o todos si hay menos de k .

Costos reales

- Push y Pop tienen costo real constante: $c_i = 1$.
- MultiPop(k) tiene costo real $c_i = \min(k, s)$, donde s es el número de elementos en la pila antes de la operación.

Definición de la función potencial

Definimos la función potencial $\Phi(D_i)$ como el número de elementos en la pila después de la i -ésima operación:

$$\Phi(D_i) = \text{número de elementos en la pila en el estado } D_i$$

Esta función cumple que $\Phi(D_i) \geq 0$ para todo i y $\Phi(D_0) = 0$ si la pila inicia vacía.

Cálculo del costo amortizado

Sea s el número de elementos en la pila antes de la i -ésima operación.

■ Push:

$$\begin{cases} c_i = 1 \\ \Phi(D_i) - \Phi(D_{i-1}) = (s+1) - s = 1 \\ \hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 + 1 = 2 \end{cases}$$

■ Pop:

$$\begin{cases} c_i = 1 \\ \Phi(D_i) - \Phi(D_{i-1}) = (s-1) - s = -1 \\ \hat{c}_i = 1 + (-1) = 0 \end{cases}$$

■ MultiPop(k): Sea $k' = \min(k, s)$

$$\begin{cases} c_i = k' \\ \Phi(D_i) - \Phi(D_{i-1}) = (s-k') - s = -k' \\ \hat{c}_i = k' + (-k') = 0 \end{cases}$$

Interpretación

- Las operaciones *Push* pagan un costo amortizado de 2, un poco mayor que su costo real, acumulando crédito en la pila.
- Las operaciones *Pop* y *MultiPop* consumen ese crédito, resultando en costo amortizado 0.
- Aunque *MultiPop* puede ser costosa en el peor caso, su costo amortizado es constante.

Conclusión

Para una secuencia de N operaciones, el costo amortizado total es

$$\sum_{i=1}^N \hat{c}_i \in O(N)$$

lo que implica un costo amortizado promedio por operación de $O(1)$, a pesar de que algunas operaciones individuales puedan ser costosas.

Este análisis muestra cómo el método del potencial distribuye el costo de operaciones caras a lo largo de operaciones más económicas, proporcionando una estimación más ajustada y útil para el análisis de estructuras de datos dinámicas.

2. Ejemplo. Inserción al Final de un Vector

2.1. Mecanismo de inserción

Suponiendo que la capacidad inicial es 1 y se duplica en cada redimensionamiento, que es lo que hace `push_back`.

- **Inserción básica:** Cuando hay espacio disponible, `push_back` inserta el elemento en tiempo $O(1)$.
- **Redimensionamiento:** Si el vector está lleno, se asigna un nuevo bloque de memoria con capacidad mayor (usualmente se duplica la capacidad), y se copian los elementos existentes, con costo $O(n)$.

2.2. Método Agregado (Análisis Global)

Premisa: Calcular el costo total de n operaciones y dividir entre n .

- Cada vez que el array se llena, se duplica su capacidad (operación costosa).
- Para n inserciones:
 - Costo real por inserción sin `resize`: 1.
 - Costo real por inserción con `resize`: $k + 1$ (donde k es el tamaño anterior).

Cálculo total:

$$T(n) = n + \sum_{j=0}^{\lfloor \log_2 n \rfloor} 2^j = n + (2^{\lfloor \log_2 n \rfloor + 1} - 1) \leq 3n$$

Costo amortizado:

$$\frac{T(n)}{n} \leq \frac{3n}{n} = O(1)$$

Clave: Los costosos `resizes` ocurren cada vez menos frecuentemente.

3. Método Contable (Créditos)

Estrategia: Cobrar “créditos extra” en operaciones simples para pagar futuros `resizes`.

Implementación:

- Haremos que cada operación `push_back` paga **3 créditos**:
 - 1 crédito para la inserción actual.
 - 2 créditos almacenados para futuras copias.

Note que hay que usar 2 créditos adicionales (no uno), ya que cada vez que se duplica el arreglo, se copian todos los elementos existentes, y cada elemento debe haber “pagado” anticipadamente su propia copia.

Dinámica durante `resize`:

- Al duplicar el array de tamaño k a $2k$:

- Costo real = k (copiar elementos).
- Se usan k créditos previamente almacenados (2 créditos $\times k/2$ inserciones).

Cuando se duplica la capacidad m , se usan los créditos acumulados para pagar la copia de m elementos, garantizando que siempre haya créditos suficientes. Así, el costo total amortizado de las n inserciones es $3n = O(n)$, y el costo amortizado por operación es $O(1)$.

4. Método Potencial

Función potencial:

$$\Phi(D_i) = 2 \times \text{size}_i - \text{capacity}_i$$

donde size es el número de elementos y capacity la capacidad actual.

Costo amortizado por operación:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

Caso 1: Inserción sin resize

$$\Delta\Phi = 2(\text{size} + 1) - \text{capacity} - (2 \times \text{size} - \text{capacity}) = 2$$

$$c_i = 1$$

$$\hat{c}_i = 1 + 2 = 3$$

Caso 2: Inserción con resize

$$c_i = k + 1 \quad (\text{copiar } k \text{ elementos} + \text{insertar})$$

$$\Delta\Phi = (2(k + 1) - 2k) - (2k - k) = 2 - k$$

$$\hat{c}_i = (k + 1) + (2 - k) = 3$$

Resultado: Costo amortizado constante $O(1)$ en ambos casos.

5. Comparación de Métodos

Método	Ventajas	Complejidad Amortizada
Agregado	Simple matemáticamente	$O(1)$
Contable	Intuitivo para diseño de estructuras	$O(1)$
Potencial	Flexible para análisis complejos	$O(1)$

Esta técnica explica por qué operaciones como `push_back` en C++ tienen comportamiento práctico eficiente a pesar de los ocasionales resizes costosos. Aunque una operación individual de `push_back` puede costar $O(n)$ debido al redimensionamiento, el costo amortizado promedio por inserción es $O(1)$.

Note además, que nuestra función potencial es siempre no negativa y comienza en un valor no negativo. Esto es necesario para garantizar que la suma de los costos amortizados sea una cota superior válida para la suma de los costos reales.

Justificación de la no negatividad de la función potencial

Considere la función potencial ya definida como:

$$\Phi(D_i) = 2 \times \text{size}_i - \text{capacity}_i$$

donde size_i es el número de elementos almacenados en el vector en el estado D_i , y capacity_i es la capacidad del vector en ese mismo estado. En un vector dinámico que duplica su capacidad cuando está lleno, se cumple que:

$$\text{capacity}_i \geq \text{size}_i$$

y justo después de una redimensión:

$$\text{capacity}_i = 2 \times \text{size}_i$$

Por lo tanto, para cualquier estado D_i :

$$\Phi(D_i) = 2 \times \text{size}_i - \text{capacity}_i \geq 2 \times \text{size}_i - 2 \times \text{size}_i = 0$$

Intuición

- Cuando el vector está lleno, $\text{capacity}_i = \text{size}_i$, por lo que

$$\Phi(D_i) = 2 \times \text{size}_i - \text{size}_i = \text{size}_i \geq 0$$

- Al realizar una redimensión, la capacidad se duplica, incrementando el valor de $\Phi(D_i)$.
- Entre redimensiones, el tamaño crece hasta alcanzar la capacidad, aumentando también $\Phi(D_i)$.

De esta forma, la función potencial representa un *crédito* acumulado que se utiliza para pagar el costo de las futuras copias durante los redimensionamientos. La función potencial $\Phi(D_i) = 2 \times \text{size}_i - \text{capacity}_i$ es siempre no negativa, garantizando así que el análisis amortizado mediante esta función es válido y que:

$$\Phi(D_i) \geq 0 \quad \forall i$$